

Playbook de Ingeniería: Fundamentos, Arquitectura y Despliegue con KerasHub, Hugging Face y LiteRT

**Guía Teórico-Práctica de Referencia para la Adopción,
Especialización y Ejecución de Modelos Fundacionales**

Audiencia objetivo: Ingenieros de Machine Learning, Arquitectos de Soluciones de IA, Tech Leads e Investigadores Aplicados.

Ámbito técnico: Ciclo de vida integral de modelos fundacionales (Visión, Lenguaje y Multimodalidad), desde la selección de pesos hasta la inferencia en producción y micro-dispositivos edge.

1. Resumen Ejecutivo y Marco Teórico Global

El paradigma contemporáneo del Deep Learning ha experimentado un cambio de fase irreversible: la transición desde el desarrollo de redes neuronales desde cero (*training from scratch*) hacia la especialización sistemática de modelos fundacionales preentrenados (*Transfer Learning*, *Parameter-Efficient Fine-Tuning* y *Edge Serving*). En este contexto, la viabilidad técnica y comercial de las iniciativas de inteligencia artificial no reside exclusivamente en la capacidad bruta de cómputo, sino en la selección rigurosa de herramientas que reduzcan la fricción en tres etapas fundamentales:

1. **Descubrimiento y Gobernanza de Modelos:** Identificación, auditoría de sesgos y gestión de dependencias criptográficas en artefactos preentrenados.
2. **Entrenamiento, Especialización y Adaptación:** Fine-tuning paramétrico y orquestación agnóstica de hardware para prevenir el bloqueo de proveedores de nube.
3. **Inferencia de Alta Eficiencia y Edge Computing:** Ejecución de modelos en entornos de recursos severamente restringidos sin dependencia de conectividad de red.

Este playbook desglosa las tres tecnologías que articulan este ciclo de vida moderno:

KerasHub como framework modular agnóstico; el **Ecosistema Hugging Face** como repositorio estándar y plataforma de entrenamiento distribuido; y **Google LiteRT / LiteRT-LM** como

runtime industrial para ejecución embebida y on-device.

2. KerasHub: Estandarización Modular y Abstracción Multi-Backend

2.1. Génesis y Contexto Arquitectónico

KerasHub representa la convergencia y unificación de dos iniciativas clave del ecosistema de Google: KerasNLP y KerasCV. Construido sobre la base de Keras 3, KerasHub está diseñado para resolver la fragmentación del código de investigación al desacoplar completamente la lógica matemática del modelo respecto al motor de cálculo subyacente.

A diferencia de marcos monolíticos donde el código de una red neuronal en PyTorch no puede ejecutarse en clústeres de TPU sin reescritura total, KerasHub compila de forma transparente hacia tres backends primarios:

- **JAX:** Máximo rendimiento a través de compilación anticipada XLA (Accelerated Linear Algebra), sharding automático de tensores y paralelismo de datos distribuido.
- **PyTorch:** Compatibilidad inmediata con el ecosistema de investigación, debugging dinámico y módulos de hardware personalizados.
- **TensorFlow:** Robustez operativa comprobada en despliegues heredados, pipelines de datos masivos con tf.data y serialización hacia SavedModel/TFLite.

2.2. Anatomía de Componentes en KerasHub

Cada arquitectura soportada en KerasHub (por ejemplo, Gemma, Llama, Whisper, Stable Diffusion, Segment Anything) se subdivide formalmente en tres niveles:

Capa / Componente	Responsabilidad Técnica	Ejemplo de Clase
1. Preprocessor	Tokenización de texto, aumentación de imágenes o extracción de espectrogramas. Reside dentro del grafo computacional para evitar fallos de preprocesamiento entre entrenamiento y serving.	keras_hub.models.GemmaCa usalLMPreprocessor

Capa / Componente	Responsabilidad Técnica	Ejemplo de Clase
2. Backbone	Extractor de representaciones latentes puro. Contiene los pesos profundos sin cabezas de clasificación ni capas de proyección específicas de tarea.	<code>keras_hub.models.GemmaBackbone</code>
3. Task Head	Capa de decisión o generación. Integra el backbone con el cabezal correspondiente (CausalLM, MaskedLM, ImageClassifier, SequenceClassifier).	<code>keras_hub.models.GemmaCausalLM</code>

2.3. Patrón de Implementación y Fine-Tuning con LoRA

KerasHub permite activar adaptadores de bajo rango (Low-Rank Adaptation - LoRA) a través de un único método declarativo sobre el backbone, congelando automáticamente los parámetros base e inyectando matrices de descomposición en las proyecciones de atención:

```
import os
os.environ["KERAS_BACKEND"] = "jax" # Selección dinámica de backend: jax, torch o tensorflow
import keras_hub

# 1. Carga declarativa de arquitectura y pesos mediante Presets
causal_lm = keras_hub.models.GemmaCausalLM.from_preset("gemma2_instruct_2b_en")

# 2. Inyección declarativa de LoRA en el extractor base
causal_lm.backbone.enable_lora(rank=8)

# 3. Verificación de reducción de parámetros entrenables
causal_lm.summary() # Los parámetros entrenables caen típicamente al ~0.5% del total

# 4. Compilación y entrenamiento estándar con optimizadores de Keras
causal_lm.compile(
    optimizer=keras.optimizers.AdamW(learning_rate=5e-5, weight_decay=0.01),
    loss=keras.losses.SparseCategoricalCrossentropy(from_logits=True)
)
```

2.4. Evaluación Crítica de KerasHub

Ventajas Operativas:

- **Portabilidad Estructural:** El desarrollador puede prototipar en PyTorch localmente y desplegar en clústeres de TPU con JAX sin modificar una sola línea de lógica de red.
- **Consistencia del Preprocesamiento:** La serialización incluye los artefactos de tokenización, eliminando el problema clásico de discrepancias en inferencia.
- **Curva de Aprendizaje Limpia:** La API conserva la elegancia y coherencia semántica clásica de Keras.

Limitaciones y Desventajas:

- **Cobertura de Modelos:** Menor velocidad de incorporación de pesos comunitarios en comparación con Hugging Face Hub.
- **Ecosistema de Serving Especializado:** Menor soporte directo para motores de inferencia de alto rendimiento como vLLM o TensorRT-LLM, requiriendo puentes de exportación.

3. El Ecosistema Hugging Face: Gobernanza y Estándar de la Industria

3.1. Arquitectura del Stack de Hugging Face

Hugging Face ha trascendido la categoría de librería para constituirse en el sistema operativo central de la comunidad de Machine Learning. Su ecosistema se compone de capas funcionales estrictamente interconectadas:

Librería / Servicio	Función en Producción	Impacto Arquitectural
Hugging Face Hub	Repositorio centralizado con control de versiones basado en Git LFS para pesos, datasets y demos.	Estandarización de Model Cards, metadata de licencias y trazabilidad de pesos.
transformers	Implementaciones canónicas de miles de arquitecturas de texto, audio, imagen y multimodales.	Adopción de "Day-0" de cualquier paper de investigación publicado globalmente.

Librería / Servicio	Función en Producción	Impacto Arquitectural
tokenizers	Motor de tokenización desarrollado en Rust optimizado para paralelismo multihilo.	Velocidades de procesamiento de gigabytes de texto en segundos; cero cuellos de botella en CPU.
PEFT & TRL	Librerías para Parameter-Efficient Fine-Tuning (LoRA, QLoRA, Prefix Tuning) y Reinforcement Learning (DPO, PPO).	Democratización del fine-tuning de modelos de 70B parámetros en GPUs de consumo comercial.
SafeTensors	Formato binario de almacenamiento de tensores seguro y eficiente con soporte para <i>memory-mapping</i> (<i>mmap</i>).	Eliminación total de vulnerabilidades de inyección de código arbitrario asociadas a Python Pickle.

3.2. Innovación en Almacenamiento: El Formato SafeTensors

Durante años, la comunidad dependió de archivos .bin o .pt generados mediante la función pickle de Python. Este mecanismo presentaba un riesgo de seguridad crítico: un actor malicioso podía incrustar código malicioso ejecutable dentro del archivo de pesos. SafeTensors resolvió este problema garantizando dos propiedades indispensables para entornos corporativos:

1. **Seguridad Criptográfica y Ausencia de Ejecución:** Almacena únicamente encabezados JSON con metadatos de dimensiones y tipo de datos, seguidos por los buffers binarios planos de los tensores.
2. **Carga por Mapeo de Memoria (Zero-Copy mmap):** Los pesos no necesitan deserializarse completamente en memoria de host para luego transferirse a la VRAM; el sistema operativo mapea el archivo directamente, reduciendo el tiempo de inicialización de modelos de minutos a milisegundos.

3.3. Evaluación Crítica de Hugging Face

Ventajas Operativas:

- Ecosistema masivo: mayor catálogo de modelos, variantes de cuantización (GGUF,

AWQ, GPTQ) y datasets a nivel mundial.

- Interoperabilidad nativa con frameworks de inferencia de alto rendimiento como vLLM, Text Generation Inference (TGI) y Triton.
- Soporte de cuantización al vuelo de 4 y 8 bits mediante bitsandbytes.

Limitaciones y Desventajas:

- **Deuda Técnica por Abstracción Excesiva:** La base de código de transformers contiene bifurcaciones complejas dentro de los métodos forward() para soportar cientos de parámetros optativos.
- **Acoplamiento Fuerte a PyTorch:** Aunque existe soporte teórico para otros backends, la gran mayoría de módulos avanzados (PEFT, TRL, Accelerate) operan con mayor estabilidad en entornos PyTorch.

4. Google LiteRT y LiteRT-LM: Inferencia Embebida y Edge AI

4.1. De TensorFlow Lite a LiteRT

LiteRT (Lite Runtime) es la evolución arquitectónica oficial de TensorFlow Lite (TFLite) desarrollada por Google AI Edge. LiteRT supera la atadura de ser un mero apéndice de TensorFlow, convirtiéndose en un runtime C++ compilado e independiente, capaz de ingerir modelos originados tanto en PyTorch (mediante ai-edge-torch) como en Keras y JAX. Su objetivo central es proveer inferencia determinista de latencia ultra-baja y consumo energético mínimo en dispositivos periféricos: smartphones Android/iOS, dispositivos IoT, procesadores automotrices y microcontroladores TinyML.

4.2. Taxonomía Funcional: LiteRT Core vs. LiteRT-LM

Con la proliferación de modelos generativos, Google bifurcó y optimizó el runtime en dos variantes especializadas:

Dimensión	LiteRT Core	LiteRT-LM (Especializado)
Tipo de Carga	Modelos de predicción estándar (CNNs, MobileNet, BERT para clasificación, Whisper audio, embeddings).	Modelos Autorregresivos y Large Language Models (Gemma 2, Llama 3, Phi-3).

Dimensión	LiteRT Core	LiteRT-LM (Especializado)
Gestión de Memoria	Asignación estática de arena de memoria; tamaño de tensores predecible.	Gestión dinámica del <i>KV-Cache</i> en buffers continuos con paginación optimizada.
Kernels Computacionales	Convoluciones 2D, Poolings, Multiplicaciones matriciales estándar.	Fusión de operadores LLM: Rotary Position Embeddings (RoPE), RMSNorm, SwiGLU y Attention decodificada.
Lógica de Muestreo (Sampling)	No aplicable (externa al runtime).	Samplers integrados en C++ (Greedy, Top-P, Top-K, Temperature) para evitar saltos entre host y runtime.

4.3. Aceleración por Hardware (Delegates)

LiteRT no ejecuta código directamente en el intérprete; descompone el grafo computacional y delega la ejecución en capas aceleradas:

- **XNNPACK (CPU Delegate):** Conjunto de microkernels altamente optimizados para arquitecturas ARM (Neon) y x86 (AVX2/AVX-512), permitiendo inferencia flotante (FP32/FP16) y entera (INT8).
- **GPU Delegate:** Acceso a hardware gráfico mediante Vulkan (Android/Linux), Metal (iOS/macOS) y OpenCL, traduciendo capas completas a compute shaders.
- **NPU / DSP Delegate:** Integración directa con Neural Processing Units de silicio dedicado (Qualcomm Hexagon, MediaTek APU, Google Tensor TPU) a través de backends especializados como QNN y Android NNAPI, alcanzando eficiencias energéticas inferiores a 1 Watt por inferencia.

5. Matriz Comparativa Exhaustiva

La siguiente tabla proporciona un análisis multidimensional de las tres tecnologías para fundamentar decisiones de arquitectura:

Criterio Técnico	KerasHub	Ecosistema Hugging Face	Google LiteRT / LiteRT-LM
Objetivo Principal	Especialización y entrenamiento modular agnóstico de backend.	Distribución, investigación abierta y entrenamiento multi-GPU.	Inferencia on-device ultra-eficiente en hardware restringido.
Backends Compatibles	JAX, PyTorch, TensorFlow.	Principalmente PyTorch (secundariamente TF/JAX).	C++ nativo (con exportadores de PyTorch, Keras, JAX).
Infraestructura Típica	Nube, Clústeres de TPUs / GPUs.	Data Centers masivos, GPUs H100/A100.	Dispositivos móviles, IoT, procesadores de borde.
Formato de Serialización	Keras Presets (.keras), SavedModel.	SafeTensors (.safetensors), PyTorch (.pt).	FlatBuffers binarios (.tflite, .lited).
Estrategias de Cuantización	Cuantización estándar de Keras / XLA.	bitsandbytes (NF4, FP4), GPTQ, AWQ.	Post-Training Quantization (PTQ) INT4/INT8, Dynamic Range.
Dependencia de Red	Requiere conexión para descarga de presets.	Requiere conexión para interacción con Hub.	Cero dependencia: Inferencia 100% offline y local.

6. Arquitectura del Pipeline Integrado de Extremo a Extremo

En arquitecturas de producción maduras, estas herramientas no actúan de manera aislada ni mutuamente excluyente. Se articulan como una cadena de valor secuencial:

1. **Fase 1 (Sourcing y Gobernanza):** Selección de pesos canónicos seguros desde Hugging Face Hub (en formato SafeTensors) y curaduría de datasets con datasets.
2. **Fase 2 (Especialización Paramétrica):** Carga del modelo en KerasHub utilizando el

backend de JAX para ejecutar LoRA Fine-Tuning con compilación XLA distribuida.

3. **Fase 3 (Optimización y Fusión de Adaptadores):** Integración de los pesos delta de LoRA en el modelo base y exportación a grafo estático.
4. **Fase 4 (Compilación hacia el Edge):** Conversión mediante ai-edge-torch o el conversor de LiteRT, aplicando cuantización entera post-entrenamiento (PTQ INT4) y generación del FlatBuffer binario.
5. **Fase 5 (Despliegue On-Device):** Carga del artefacto .litedt en aplicaciones cliente (Android/iOS) con aceleración por GPU/NPU mediante LiteRT-LM.

6.1. Ejemplo de Conversión hacia LiteRT (PyTorch / Edge)

```
import torch
import ai_edge_torch

# Supongamos un modelo ajustado proveniente de PyTorch / Hugging Face
model = torch.load("fine_tuned_backbone.pt").eval()
sample_inputs = (torch.randn(1, 224, 224, 3),)

# Compilación y conversión directa a formato LiteRT optimizado
edge_model = ai_edge_torch.convert(model, sample_inputs)

# Exportación del FlatBuffer listo para ejecución en NPU móvil
edge_model.export("model_optimized.tflite")
```

7. Guía de Prevención de Errores Críticos y Anti-Patronos

- **Olvido Catastrófico (Catastrophic Forgetting):** Se produce cuando se realiza fine-tuning completo con una tasa de aprendizaje excesivamente alta (ej. $1e-3$), destruyendo los pesos preentrenados. *Solución:* Congelar el backbone durante las primeras épocas (Head Warm-up) y reducir la tasa a rangos de $1e-5$ o utilizar exclusivamente adaptadores LoRA.
- **Discrepancia de Preprocesamiento (Preprocessing Mismatch):** Ocurre cuando el modelo es alimentado con imágenes normalizadas en rangos $[0, 1]$ cuando fue preentrenado con rangos $[-1, 1]$ o con medias ImageNet. *Solución:* Empaquetar siempre las capas de preprocesamiento dentro del artefacto serializado, tal como prescribe KerasHub.
- **Desbordamiento de Memoria por Fugas en el KV-Cache:** En aplicaciones LLM on-device, omitir la gestión de contexto estático agota la memoria unificada del

smartphone provocando cierres inesperados (OOM crashes). *Solución:* Emplear LiteRT-LM con límites fijos de longitud de contexto y cuantización de KV-cache en INT8.

8. Referencias Bibliográficas y Documentales

1. **Chollet, F. et al. (2023).** *Keras 3: Universal Deep Learning for Python, JAX, PyTorch, and TensorFlow*. Documentación técnica de arquitectura.
2. **Wolf, T. et al. (2020).** *Transformers: State-of-the-Art Natural Language Processing*. Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations (EMNLP).
3. **Hu, E. J. et al. (2021).** *LoRA: Low-Rank Adaptation of Large Language Models*. arXiv preprint arXiv:2106.09685.
4. **David, R. et al. (2021).** *TensorFlow Lite Micro: Embedded Machine Learning on TinyML Systems*. Proceedings of Machine Learning and Systems (MLSys).
5. **Google AI Edge Team (2024).** *LiteRT Architecture and On-Device Generative AI Execution*. Guías oficiales de desarrollo para Android y Edge Systems.